

Review of Polkadot Architecture

Polkadot is designed as a heterogeneous multi-chain ecosystem that emphasizes interoperability, shared security, and scalability among interconnected blockchains. At the core of this framework is the **relay chain**, serving as the primary coordinator. This chain maintains minimal functionality to reduce complexity; it does not natively support smart contracts and primarily focuses on governance, staking via Nominated Proof-of-Stake (NPoS), and interaction coordination. Validators on the relay chain stake DOT tokens, produce blocks, and validate proofs from connected chains using the Availability & Validity (AnV) protocol to ensure block availability and safeguard against data breaches that could lead to withholding.

Connected to the relay chain are **parachains** (dedicated slots) and **on-demand parachains** (formerly parathreads), which handle specialized computations. The relay chain supports a fixed number of execution cores (e.g., CPU cores), enabling parallel processing. Parachains delegate computations away from the relay chain, enabling diverse use cases such as smart contracts and privacy-focused chains. **Bridges** extend this by connecting external standalone chains (e.g., Ethereum or Bitcoin), facilitating arbitrary data transfers while maintaining security through the relay chain's consensus.

The **shared security model** is an essential characteristic: all interconnected parachains benefit from the validator security of the relay chain, which staked DOT supports. This consolidated security architecture eliminates the need for each parachain to establish its own validators, thereby reducing risks such as 51% attacks commonly associated with sovereign chains. If the relay chain reverts a block, all parachains revert accordingly, thereby ensuring system-wide integrity consistency.

Scalability is attained through parallel processing across cores, adaptable resource allocation facilitated by DOT-based coretime purchases, and the delegation of workloads. This approach differentiates from monolithic blockchains, thereby enabling the network to manage high transaction volumes without central overload. As of October 2025, no significant architectural updates have been observed beyond the implementation of Agile Coretime, which enhances resource management.

Review of Parachain Mechanics

Parachains are application-specific data structures that connect to the relay chain for shared security and interoperability. They operate as deterministic state machines, storing state in a Merkle tree and executing transitions via WebAssembly (WASM) executables. Parachains connect via **cores** on the relay chain, purchased with DOT coretime for either dedicated (continuous) or on-demand (intermittent) access. System parachains (e.g., for bridges) are allocated via governance, while others use bulk or on-demand purchases—DOT is burned in the process.

Slot auctions and crowdloans have been deprecated since the activation of Agile Coretime; existing leases are automatically converted to coretime. In the absence of coretime, a parachain becomes a parathread, retaining its ParaID but relinquishing active security participation.

Block production involves **collators**, who operate full nodes on both the parachain and relay chain. They are responsible for collecting transactions, producing block candidates, and submitting Proof-of-Validity (PoV) blocks to validators. Validators then verify these blocks against the parachain's state transition function (STF), which is stored on the relay chain. The

parachain host incorporates runtime modules for logic and node-side subsystems (such as backing and availability), which are managed by an Overseer responsible for overseeing these process forks.

Validators offer pooled security through the relay chain, incentivized via NPoS, whereas collators' rewards are specific to the parachain, such as through native tokens or inflation. Parachains have the ability to establish customized economic models, including Proof-of-Stake, but depend on relay chain protocols for validation, such as meeting minimum requirements fees.

Scalability is achieved through parallel processing, facilitating the deployment of specialized features without affecting the entire network. **Interoperability** is managed via XCM messaging, which introduces at least two blocks of latency. Hubs, such as Asset Hub, enable more rapid composability within sectors like DeFi. Relevant use cases include privacy chains and smart contract platforms. As of October 2025, coretime testing is accessible on Paseo, and Polimec provides support for decentralized operations fundraising.

Review of ink! Smart Contracts

ink! is a Rust-based domain-specific language designed for developing WASM smart contracts on Substrate-based chains such as Polkadot and Kusama. It exploits Rust's safety features, including memory safety and the absence of undefined behavior, to produce secure and efficient contracts. These contracts compile to WebAssembly (WASM) for execution, utilizing **PolkaVM**, a RISC-V engine, to ensure high performance and minimal gas consumption costs.

Features include a composable, modular design that facilitates component reuse; ensures interoperability within the Polkadot ecosystem; and maintains native compatibility with Solidity and tools such as MetaMask, thereby bridging EVM and non-EVM environments. This enables Solidity developers to use familiar tooling while also leveraging Rust's security capabilities.

Benefits over Solidity (Ethereum's language) include Rust's built-in defenses against vulnerabilities, enhanced performance via PolkaVM, and lower transaction fees. ink! circumvents the limitations of Ethereum's gas model, focusing on efficiency.

Integration with Substrate and Polkadot: ink! contracts are deployed to parachains and interact with the Substrate runtime for efficient state management. They facilitate robust, future-proof cross-chain communication.

Development process encompasses building, testing (e.g., unit tests), and deployment, using tools such as cargo-contract. Examples and tutorials range from fundamental to advanced levels of composability.

Security considerations: Rust's guarantees minimize risks such as reentrancy; additional tools, like fuzzing, enhance audits.

As of October 2025, version 6 (v6) is the current version, with Solidity/MetaMask compatibility as a key addition.

Review of XCM Protocol Specifications

XCM (Cross-Consensus Messaging) constitutes a messaging format rather than a protocol, designed for intent-based communication between consensus systems to promote interoperability within Polkadot and beyond. It facilitates chains in transmitting actions (e.g., asset transfers)

without presuming a specific consensus mechanism, thereby supporting asynchronous, absolute, asymmetric, and agnostic principles: messages are autonomous, assuredly accurate, unidirectional, and universally applicable.

Message formats utilize instructions such as `WithdrawAsset` (to remove assets to a holding register), `BuyExecution` (to pay fees based on weight), and `DepositAsset` (to transfer assets to the beneficiary). `MultiLocations` specify entities (e.g., accounts, chains) hierarchically; `MultiAssets` describe fungibles and NFTs. For example, transferring tokens involves withdrawing, executing a purchase, and depositing, with the details intricately defined.

Asset transfers: Utilize instructions for the sender to withdraw, execute fees, and deposit to the recipient, with specified weight limits to facilitate control costs.

Remote execution: Messages delineate the intents for remote chains to interpret, thereby facilitating function calls, asset locking, and NFT operations. Programmability encompasses branching and safety features dispatches.

Cross-chain communication: Facilitated through channels with bridging capabilities for multi-hop transfers (e.g., Polkadot relaying to Ethereum). Asynchronous processing ensures independence.

Interoperability: Connects parachains and external chains via universal references, supporting token transfers and state references. Last updated August 15, 2025; no new versions noted by October 2025.

How These Elements Fit into the Project

The project, a cross-chain content rights management service designed for decentralized subscription, pay-per-view (PPV), and purchase models, leverages Polkadot's architecture to ensure shared security and scalability, thereby surpassing single-chain alternatives such as Ethereum. The relay chain provides pooled validation, ensuring secure, cost-effective operations across parachains, while bridges facilitate Ethereum integration to enhance interoperability.

Parachain mechanisms enable the transition of prototypes to designated slots for handling high-volume transactions, such as thousands of subscriptions, by utilizing coretime to achieve effective scalability. Collators supervise content rights blocks, with validators verifying Proofs of Verification (PoVs) to ensure integrity and prevent tampering monetization.

The ink! smart contracts are essential components: `SubscriptionManager.rs` oversees recurring access, `PayPerView.rs` manages one-time tokens, and `PurchaseVerification.rs` facilitates ownership transfers. The security features inherent in Rust and the high efficiency of WebAssembly (WASM) are well aligned with the project's low-latency goals, with integration accomplished through Substrate chain extensions for state management.

XCM facilitates seamless cross-chain rights transfers, such as subscription activations between parachains or via Ethereum bridges, using instructions for asset movement and remote execution. This approach addresses the research question on interoperability, thereby promoting economic efficiency by reducing fees and enabling transparent revenue management and sharing.

Overall, these components establish a scalable and interoperable framework that enables creators, as delineated in the methodology and evaluation phases.